

## Assignment No. 2

### Title

A star (A\*) Algorithm for any game search problem.

### Aim

Implement A star (A\*) Algorithm for any game search problem.

### Objectives

1. To understand the concept of informal search
2. To implement A\* Algorithm using BFS.

### Theory

#### Abstract Data Types

##### What is A\* Algorithm?

**A \* algorithm** is a searching algorithm that searches for the shortest path between the *initial and the final state*. It is used in various applications, such as *maps*.

In *maps* the A\* algorithm is used to calculate the shortest distance between the source (initial state) and the destination (final state). The most important advantage of A\* search algorithm which separates it from other traversal techniques is that it has a brain. This makes A\* very smart and pushes it much ahead of other conventional algorithms

##### How it works

A\* algorithm has 3 parameters:

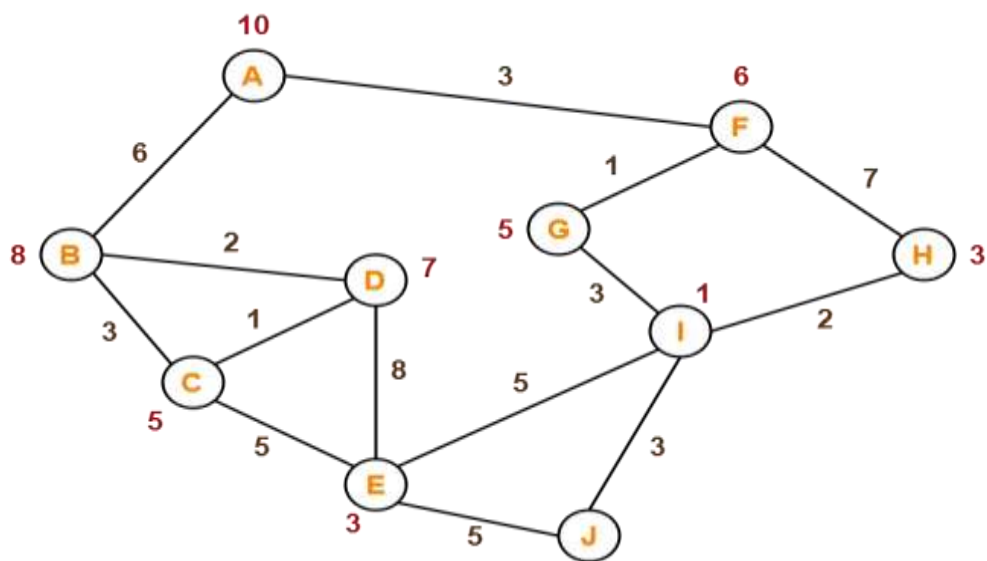
- **g**: the cost of moving from the initial cell to the current cell. Basically, it is the sum of all the cells that have been visited since leaving the first cell.
- **h**: also known as the *heuristic value*, it is the **estimated** cost of moving from the current cell to the final cell. The actual cost cannot be calculated until the final cell is reached. Hence, h is the estimated cost. We **must** make sure that there is **never** an over estimation

- **f**: it is the sum of g and h. So,  $f = g + h$

The way that the algorithm makes its decisions is by taking the f-value into account. The algorithm selects the *smallest f-valued cell* and moves to that cell. This process continues until the algorithm reaches its goal cell.

A\* Algorithm works by vertices in the graph which start with the starting point of the object and then repeatedly examines the next unexamined vertex, adding its vertices to the set of vertices that will be examined. A\* Algorithm is popular because it is a technique that is used for finding path and graph traversals. This algorithm is used by many web-based maps and games.

Example:



The numbers written on edges represent the distance between the nodes.

The numbers written on nodes represent the heuristic value.

Find the most cost-effective path to reach from start state A to final state J using A\* Algorithm.

Solution-

**Step-01:**

We start with node A. Node B and Node F can be reached from node A. A\* Algorithm calculates  $f(B)$  and  $f(F)$ .

$$f(B) = 6 + 8 = 14$$

$$f(F) = 3 + 6 = 9$$

Since  $f(F) < f(B)$ , so it decides to go to node F.

Path-  $A \rightarrow F$

**Step-02:**

Node G and Node H can be reached from node F.

A\* Algorithm calculates  $f(G)$  and  $f(H)$ .

$$f(G) = (3+1) + 5 = 9$$

$$f(H) = (3+7) + 3 = 13$$

Since  $f(G) < f(H)$ , so it decides to go to node G.

Path-  $A \rightarrow F \rightarrow G$

### Step-03:

Node I can be reached from node G.

A\* Algorithm calculates  $f(I)$ .

$$f(I) = (3+1+3) + 1 = 8$$

It decides to go to node I.

Path-  $A \rightarrow F \rightarrow G \rightarrow I$

### Step-04:

Node E, Node H and Node J can be reached from node I.

A\* Algorithm calculates  $f(E)$ ,  $f(H)$  and  $f(J)$ .

$$f(E) = (3+1+3+5) + 3 = 15$$

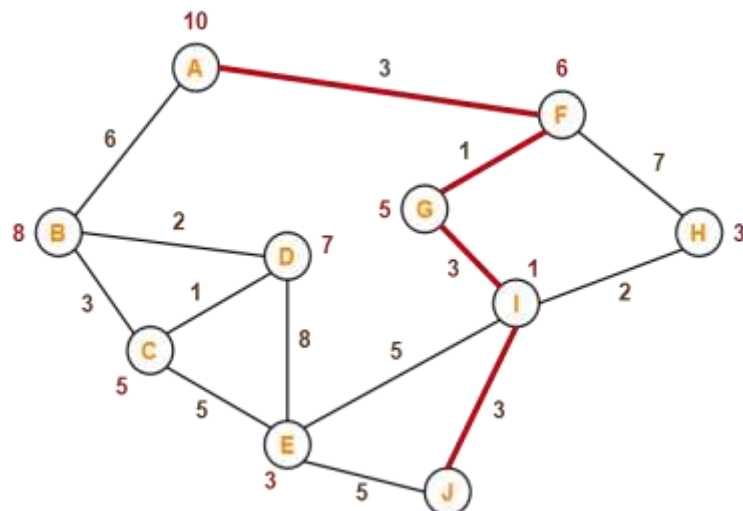
$$f(H) = (3+1+3+2) + 3 = 12$$

$$f(J) = (3+1+3+3) + 0 = 10$$

Since  $f(J)$  is least, so it decides to go to node J.

Path-  $A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$

This is the required shortest path from node A to node J.



## A\* Algorithm

**Step 1:** Place the starting node in the OPEN list.

**Step 2:** Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

**Step 3:** Select the node from the OPEN list which has the smallest value of evaluation function( $g+h$ ), if node  $n$  is goal node then return success and stop, otherwise

**Step 4:** Expand node  $n$  and generate all of its successors, and put  $n$  into the closed list. For each successor  $n'$ , check whether  $n'$  is already in the OPEN or CLOSED list, if not then compute evaluation function for  $n'$  and place into Open list.

**Step 5:** Else if node  $n'$  is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest  $g(n')$  value.

**Step 6:** Return to Step 2

## Advantages:

1. A\* search algorithm is the best algorithm than other search algorithms.
2. A\* search algorithm is optimal and complete.
3. This algorithm can solve very complex problems

## Disadvantages:

1. It does not always produce the shortest path as it mostly based on heuristics and approximation.
2. A\* search algorithm has some complexity issues.
3. The main drawback of A\* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

## Conclusion:

Hence, we have implemented A\* Algorithm for any game search problem.

## Questions:

- 1) What is A\* Algorithm? How does it work?
- 2) Why is A\* algorithm popular?
- 3) What is a Heuristic Function?
- 4) What is significance of Heuristics Function in Uninformed search?